

UNIVERSIDADE DO VALE DO ITAJAÍ

Escola Politécnica

Ciência da Computação

Segurança de Sistemas Computacionais

Case Prático 1: Avaliação de Vulnerabilidades em Aplicação Web

Cenário A, OWASP Juice Shop

Diego Fonseca da Silva

Leonardo Pacheco B Dias

Gustavo Gonçalves Trindade

João Victor Rodrigues Santos

Professor: Ramicés dos Santos Silva

Itajaí, setembro de 2026

Escopo e conduta ética

O alvo avaliado neste trabalho é o OWASP Juice Shop, aplicação web propositalmente vulnerável, mantida pela comunidade OWASP para fins educacionais. Toda a exploração ocorreu exclusivamente contra a instância local do próprio grupo, executada em contêiner Docker e acessível apenas em 127.0.0.1, sem exposição à rede externa. Nenhum teste foi realizado contra sistema de terceiros, o que manteria o grupo fora do amparo da Lei nº 12.737/2012 caso ocorresse.

Todos os caminhos citados neste projeto referenciam do repositório <https://github.com/ThreeOneOneZero/vulnerable-web-application/tree/dev-diego>.

1. Introdução

A avaliação identificou sete vulnerabilidades confirmadas no Juice Shop, distribuídas em seis categorias do OWASP Top 10:2021: A01 (Broken Access Control), A02 (Cryptographic Failures), A03 (Injection), A04 (Insecure Design), A05 (Security Misconfiguration, com duas ocorrências distintas) e A07 (Identification and Authentication Failures). As três categorias obrigatórias definidas no roteiro do trabalho, A01, A02 e A03, foram cobertas com evidência reproduzível, e a cobertura mínima de seis categorias distintas exigida pelo roteiro está atendida.

As duas vulnerabilidades de maior severidade são críticas (CVSS 9.1): uma injeção SQL no formulário de login, que permite autenticação como administrador sem conhecimento da senha, e uma falha no fluxo de recuperação de senha por pergunta de segurança, que permite assumir o controle completo de uma conta testando um pequeno conjunto de respostas prováveis, sem qualquer bloqueio. Em seguida, a listagem de diretório em /ftp/ expõe um documento interno marcado como confidencial (CVSS 7.5, alto). As demais vulnerabilidades, exposição do hash de senha no token de autenticação, quebra de controle de acesso na cesta de compras, ausência de bloqueio contra força bruta no login e página de erro que revela a versão do framework do servidor, têm severidade média.

A categoria A06 (Vulnerable and Outdated Components) foi investigada a partir da versão do Express exposta na página de erro, mas não gerou achado: a

versão identificada já corrige os CVEs mais recentes conhecidos do framework. Essa investigação está descrita na Seção 7.4 e não conta para o mínimo de seis categorias, por não ter produzido achado confirmado. As categorias A08, A09 e A10 permanecem sem investigação.

2. Configuração do ambiente (Docker)

O alvo é instanciado por meio de um único serviço no docker-compose.yml, com a porta do contêiner ligada exclusivamente em 127.0.0.1, de modo que a aplicação vulnerável não fica acessível a partir de outro dispositivo da rede.

```
docker compose up -d    # sobe o Juice Shop
# acesso: http://127.0.0.1:3000
docker compose down     # derruba o ambiente
```

A imagem utilizada é bkimminich/juice-shop:v20.2.0, com tag fixa em vez de latest, o que garante que o ambiente permaneça o mesmo em qualquer máquina e em qualquer data de execução. O contêiner possui verificação de integridade (healthcheck) configurada, de forma que docker compose ps reporta o estado healthy assim que a aplicação está pronta para uso. A porta de host é configurável por meio de um arquivo .env (modelo em .env.example), recurso incluído para que um conflito de porta em outra máquina da equipe não exija alterar o docker-compose.yml.

A reprodutibilidade do ambiente foi verificada derrubando e recriando o contêiner do zero (docker compose down seguido de docker compose up -d), com retorno ao mesmo estado íntegro em todas as execuções realizadas durante o trabalho.

3. Papéis da equipe

Integrante	Responsabilidade
Leonardo Pacheco B Dias	Subida inicial do ambiente Docker; identificação e evidência original dos achados obrigatórios A01, A02 e A03 (V1 a V3); redação do relatório
Diego Fonseca da Silva	Reprodutibilidade do ambiente Docker (tag fixa, healthcheck, porta configurável); scripts de prova de conceito de V1 a V7; identificação, evidência e classificação de V4 a V7 (A05, A07, A04); validação de CWE na base do MITRE e justificativa de CVSS métrica a métrica para todos os achados; estruturação das evidências em pastas por achado; redação do relatório
Gustavo Gonçalves Trindade	Revisão de código, testes, revisão do relatório e apoio na coleta de evidências das vulnerabilidades

João Victor Rodrigues Santos	Revisão de código, testes e revisão do relatório
------------------------------	--

4. Arquitetura do laboratório

O laboratório é composto por um único contêiner, executando o Juice Shop sobre Node.js e Express, isolado em uma rede Docker do tipo bridge. A máquina de teste é o próprio host local, com navegador e Postman usados para inspecionar e reproduzir as requisições.

```
[ Navegador + Postman ] --HTTP--> [ 127.0.0.1:3000 ]
      host local                contêiner juice-shop
                                rede labnet (172.19.0.0/16)
```

5. Endereços de rede utilizados

Item	Valor
Alvo (URL)	http://127.0.0.1:3000
Porta mapeada	127.0.0.1:3000:3000 (host para contêiner)
Rede Docker	labnet (bridge, isolada)
Sub-rede	172.19.0.0/16
Máquina de teste	host local

Endereço e sub-rede obtidos com:

```
docker network inspect vulnerable-web-application_labnet
```

6. Ferramentas de apoio utilizadas

Ferramenta	Uso neste trabalho
Docker e Docker Compose	Subida e isolamento do alvo vulnerável
Navegador e DevTools (Network, Console)	Inspeção de requisições, cookies e token JWT; execução da prova de conceito original de V3
Postman	Reprodução manual das requisições de V4 e V5, com captura de tela da resposta renderizada
jwt.io	Decodificação visual do JWT da conta administrativa (V2)
md5sum	Confirmação de que o hash vazado corresponde ao MD5 de uma senha em texto claro
sqlmap, via Docker (googlesky/sqlmap)	Confirmação automatizada da injeção booleana cega no login (V1) e identificação do SGBD como SQLite
OWASP ZAP, via Docker (zap-baseline.py, imagem oficial zapoxy/zap-stable)	Varredura passiva e spider inicial, que sinalizou a ausência de CSP posteriormente aprofundada em V4

Scripts Python (scripts/v1_sql_login.py, v2_hash_senha_jwt.py, v3_idor_cesta.py)	Reprodução determinística de V1 a V3, com gravação automática de requisição e resposta
curl	Captura de evidência textual de V4 e V5, e requisição e resposta complementares de V1 a V3

O uso interativo do OWASP ZAP (proxy configurado no navegador, Active Scan dirigido, HUD) e do Burp Suite Community (Proxy, Repeater) depende de interface gráfica operada manualmente e não foi empregado além do já descrito nesta seção.

7. Achados

Cada achado seguiu o mesmo ciclo: reprodução manual do comportamento, classificação em OWASP Top 10 e CWE, pontuação em CVSS e análise de impacto na tríade de confidencialidade, integridade e disponibilidade, além das implicações sob a LGPD. O detalhamento completo de cada achado, incluindo pré-condições, passos, payload, requisição e resposta na íntegra, CWE validado na base do MITRE, vetor CVSS justificado métrica a métrica e recomendação de mitigação, está em tabela-de-achados.md. Esta seção percorre a evidência visual de cada um.

Os sete vetores CVSS foram conferidos na calculadora interativa do FIRST (first.org/cvss/calculator/3.1), com o score exibido coincidindo com o calculado manualmente em todos os casos. Como V1 e V7 compartilham o mesmo vetor, assim como V2 (na leitura isolada) e V3, e V2 (na leitura encadeada) e V5, cinco capturas de tela (evidencias/cvss_p1.png, cvss_p2.png, cvss_v2_encadeado.png, cvss_v4.png, cvss_v6.png) cobrem a validação dos sete achados, sem repetir uma captura para vetores idênticos.

Faixas CVSS: None 0.0, Low 0.1–3.9, Medium 4.0–6.9, High 7.0–8.9, Critical 9.0–10.0. Calculadora oficial: first.org/cvss/calculator/3.1. Os vetores CVSS completos e as evidências reprodutíveis de cada achado estão detalhados nas subseções 7.1 a 7.8, a seguir.

ID	Achado	OWASP	CWE	Severidade	Impacto (CID / LGPD)
V1	SQLi no login (' OR 1=1-')	A03	CWE-89	9.1 Crítico	Confid. e integr.; risco de violação em massa
V2	Senha (hash MD5) exposta no payload do JWT	A02	CWE-522, CWE-916	6.5 Médio *	Confid.; credencial de autenticação

V3	IDOR na cesta (/rest/basket/{id})	A01	CWE-639	6.5 Médio	Confid.; dado pessoal de terceiro
V4	Erro verboso expõe framework/versão; CSP/HSTS ausentes	A05	CWE-209, CWE-16	5.3 Médio	Confid. (reconhecimento)
V5	Listagem de diretório em /ftp/ expõe doc. confidencial	A05	CWE-548	7.5 Alto	Confid.
V6	Login sem bloqueio contra força bruta	A07	CWE-307	6.5 Médio	Confid. e integr.; acesso indevido à conta
V7	Redefinição de senha via pergunta de segurança sem limite de tentativas	A04	CWE-640	9.1 Crítico	Confid. e integr.; takeover completo da conta

* Nota de metodologia sobre V2: o vetor da tabela (6.5) pontua o achado avaliado isoladamente, exigindo uma conta própria (PR:L) para obter e decodificar um token. O mesmo achado, encadeado a partir do bypass de SQL Injection de V1, dispensa qualquer conta própria (PR:N) e eleva o vetor para 7.5, o que agrava o risco combinado na prática. A prática recomendada de CVSS de pontuar cada vulnerabilidade por seus próprios pré-requisitos mínimos, sem presumir outra exploração prévia, justifica adotar 6.5 como vetor de referência de V2 neste relatório.

Cobertura mínima do roteiro (seis categorias distintas, com A01, A02 e A03 obrigatórias): atingida. V1 a V7 cobrem seis categorias distintas (A01, A02, A03, A04, A05 e A07), com as três obrigatórias confirmadas. A06 foi investigada e não gerou achado (Seção 7.4); A08, A09 e A10 ainda não foram investigadas.

7.1 V1, SQL Injection no login (A03)

O campo de e-mail do formulário de login aceita o payload ' OR 1=1--, que comenta a verificação de senha na consulta executada pelo servidor e autentica o requisitante como o primeiro registro da tabela de usuários, a própria conta administrativa.

A Figura 1 mostra a requisição de login capturada no DevTools, com o payload no corpo em formato JSON.

Arquivo	Iniciador	Tipo	Transferido	Ta...	Cabeçalhos	Cookies	Requisição
whoami	polyfills.js2 (fetch)	json	394 B	1...	✓ Filtrar parâmetros da requisição		
login	polyfills.js2 (fetch)	json	1,17 kB	7...	JSON		
1	polyfills.js2 (fetch)	json	801 B	1...	email: "" OR 1=1--"		
whoami?fields=email	polyfills.js2 (fetch)	json	422 B	3...	password: "x"		

Figura 1 – Requisição de login capturada no DevTools, com o payload no corpo em formato JSON.

A Figura 2 mostra a resposta correspondente, contendo o token de autenticação e o campo umail já preenchido com admin@juice-sh.op.

Arquivo	Iniciador	Tipo	Transferi...	Ta	Cabeçalhos	Cookies	Requisição	Resposta	Tempos	Stack Trace
whoami	polyfills.js2 (f...	json	394 B	11	✓ Filtrar propriedades					
login	polyfills.js2 (f...	json	1,17 kB	78	JSON					
1	polyfills.js2 (f...	json	801 B	1,3	authentication: { token:					
whoami?fields=email	polyfills.js2 (f...	json	422 B	38						
whoami	polyfills.js2 (f...	json	510 B	12						
/api/Quantities/	polyfills.js2 (f...	json	em cache	7,6						
search?q=	polyfills.js2 (f...	json	em cache	16						
continue-code	polyfills.js2 (f...	json	463 B	79						
confetti-DoPrSMNP.js	main.js:3498 ...	js	em cache	11						
apple_juice.jpg	img	jpeg	em cache	12						
apple_pressings.jpg	img	jpeg	em cache	20						

Figura 2 – Resposta do login, contendo o token de autenticação e o campo umail já preenchido com admin@juice-sh.op.

A Figura 3 confirma o resultado no próprio navegador: o menu de conta exibe a sessão autenticada como administrador, sem que a senha real tenha sido informada em nenhum momento.

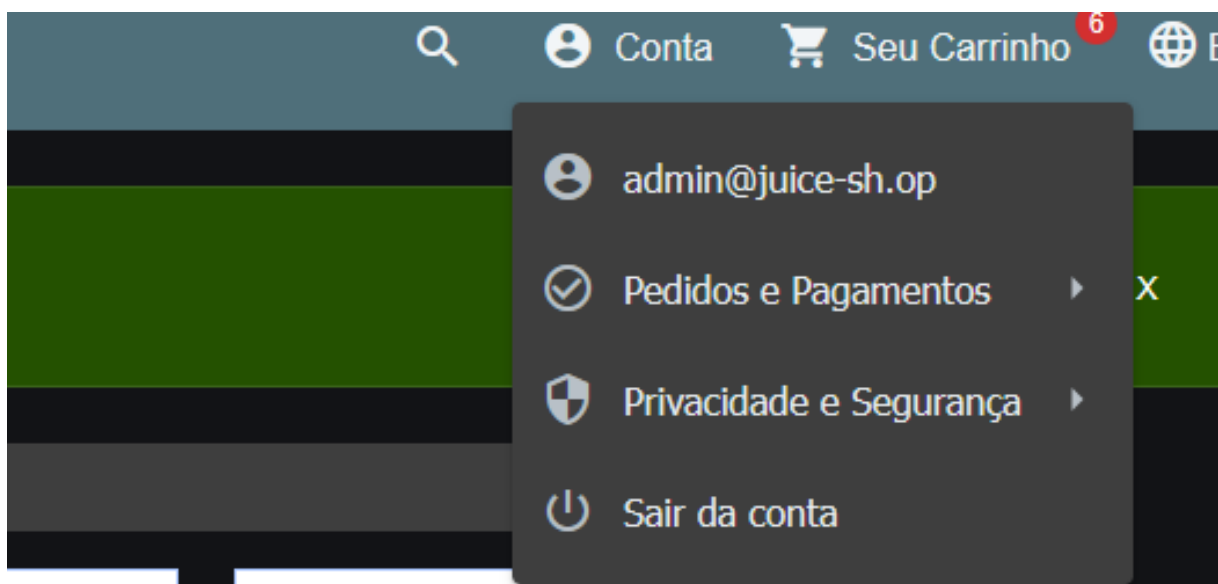


Figura 3 – Menu de conta no navegador, sessão autenticada como administrador após o bypass de SQL Injection.

A mesma falha foi reproduzida de forma independente com o script scripts/v1_sql_login.py, cuja requisição e resposta estão salvas em evidencias/V1-sqli-

análise da decodificação estão em evidencias/V2-hash-senha-jwt/http/ e evidencias/V2-hash-senha-jwt/analise-jwt.txt.

7.3 V3, controle de acesso quebrado na cesta de compras (A01)

Uma conta autenticada consegue ler a cesta de compras de outra conta apenas trocando o identificador numérico na URL da requisição, sem que o servidor verifique se o token pertence ao dono do recurso solicitado.

A Figura 5 mostra o resultado obtido no console do navegador, autenticado como um usuário, ao requisitar diretamente a cesta de identificador diferente do próprio: a resposta retorna com sucesso e traz produtos e um UserId que não corresponde à conta usada na requisição.



Figura 5 – Resultado no console do navegador ao requisitar a cesta de outro usuário: a resposta retorna com sucesso e traz um UserId que não corresponde à conta autenticada.

A prova de conceito complementar, em scripts/v3_idor_cesta.py, elimina a dependência de uma sessão logada manualmente: registra duas contas de teste isoladas, adiciona um produto à cesta da primeira e, em seguida, usa apenas o token da segunda para ler a cesta da primeira. A sequência completa de seis requisições e respostas está em evidencias/V3-idor-cesta/http/.

7.4 V4, página de erro verbosa e ausência de cabeçalhos de segurança (A05)

Um cabeçalho Authorization fora do formato esperado provoca uma página de erro completa, em vez de uma mensagem genérica, revelando o nome e a versão do framework do servidor. Em nenhuma resposta testada aparecem os cabeçalhos Content-Security-Policy ou Strict-Transport-Security.

A Figura 6 mostra, no Postman, a requisição com o cabeçalho Authorization: BearerSemEspaco e a resposta renderizada em modo Preview, com o texto OWASP Juice Shop (Express ^4.22.1) em destaque.

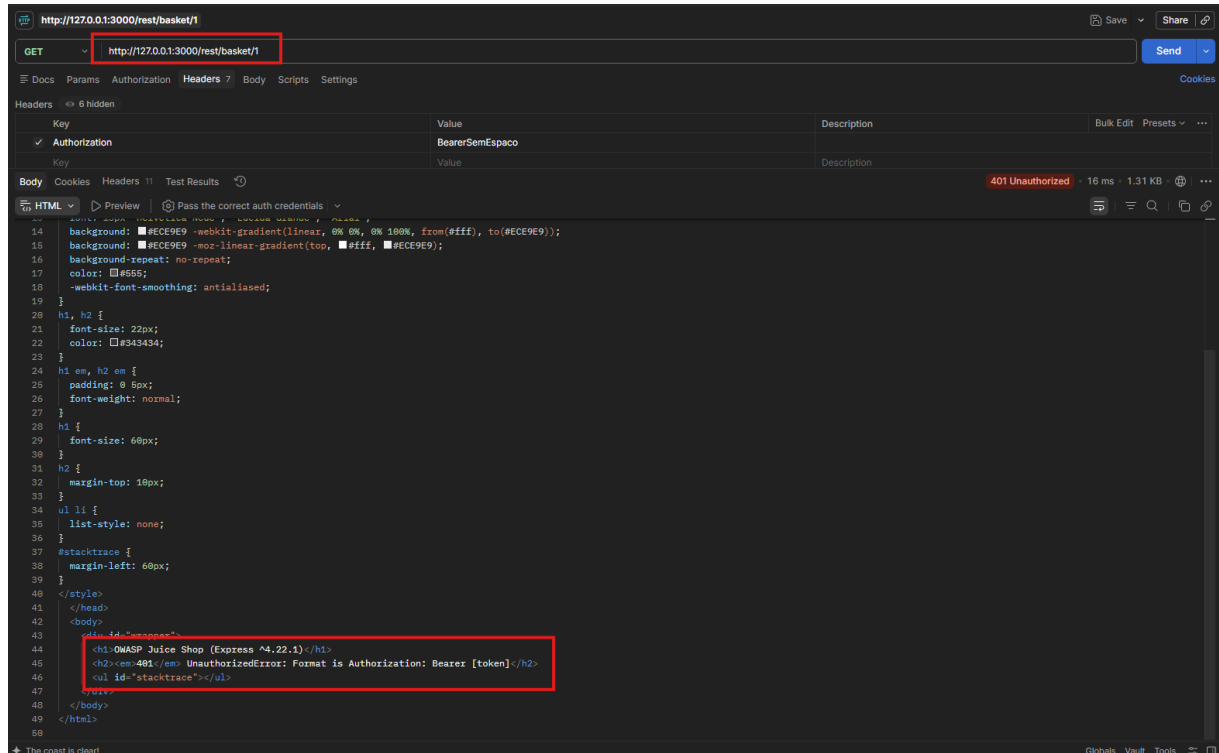


Figura 6 – Requisição com o cabeçalho Authorization: BearerSemEspaco no Postman e a resposta renderizada, com a versão do Express em destaque.

A Figura 7 mostra a lista completa de cabeçalhos da resposta da página inicial, na qual não aparece nenhum cabeçalho de política de segurança.

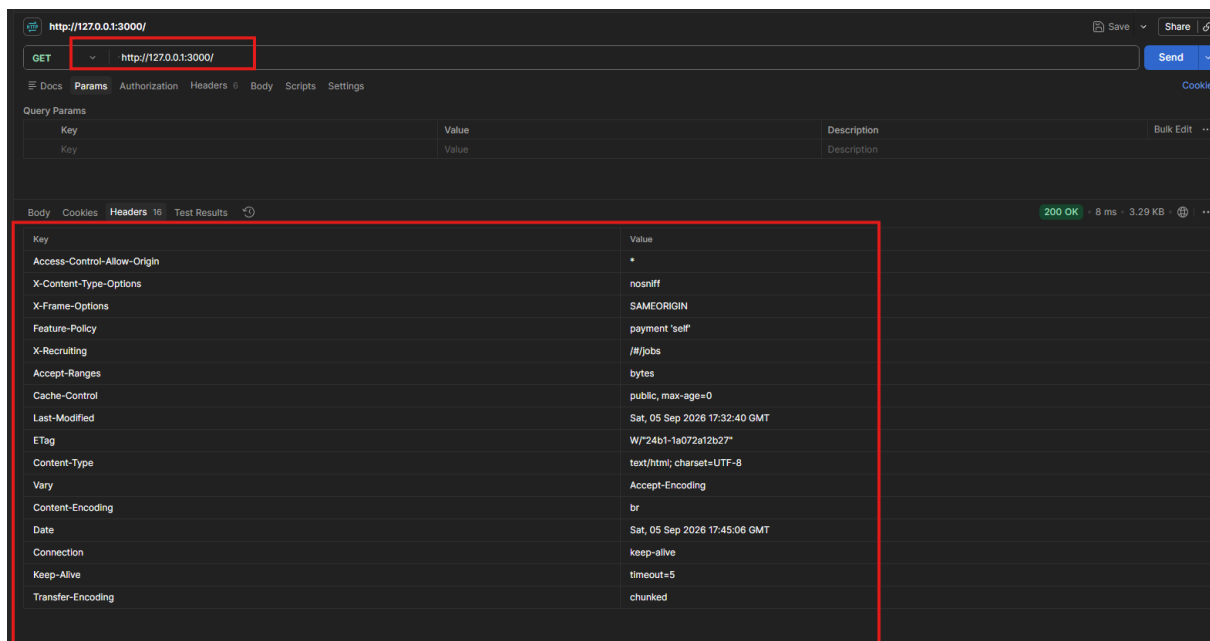


Figura 7 – Cabeçalhos de resposta da página inicial, sem Content-Security-Policy nem Strict-Transport-Security.

A versão do Express identificada nessa página de erro foi correlacionada com a base de vulnerabilidades conhecidas do framework: a versão 4.22.1 já corrige os problemas mais recentes relevantes (redirecionamento aberto, XSS refletido, negação de serviço por expressão regular e poluição de protótipo via analisador de query), todos corrigidos até a versão 4.22.0. Por esse motivo, a categoria A06 foi investigada, sem produzir achado confirmado.

7.5 V5, listagem de diretório expõe documento confidencial (A05)

O caminho `/ftp/` retorna uma listagem completa dos arquivos armazenados no servidor, sem exigir autenticação. Entre os arquivos listados, ao menos três são baixáveis livremente, um deles contendo, no próprio texto, a frase "This document is confidential! Do not distribute!".

A Figura 8 mostra a listagem renderizada no Postman.

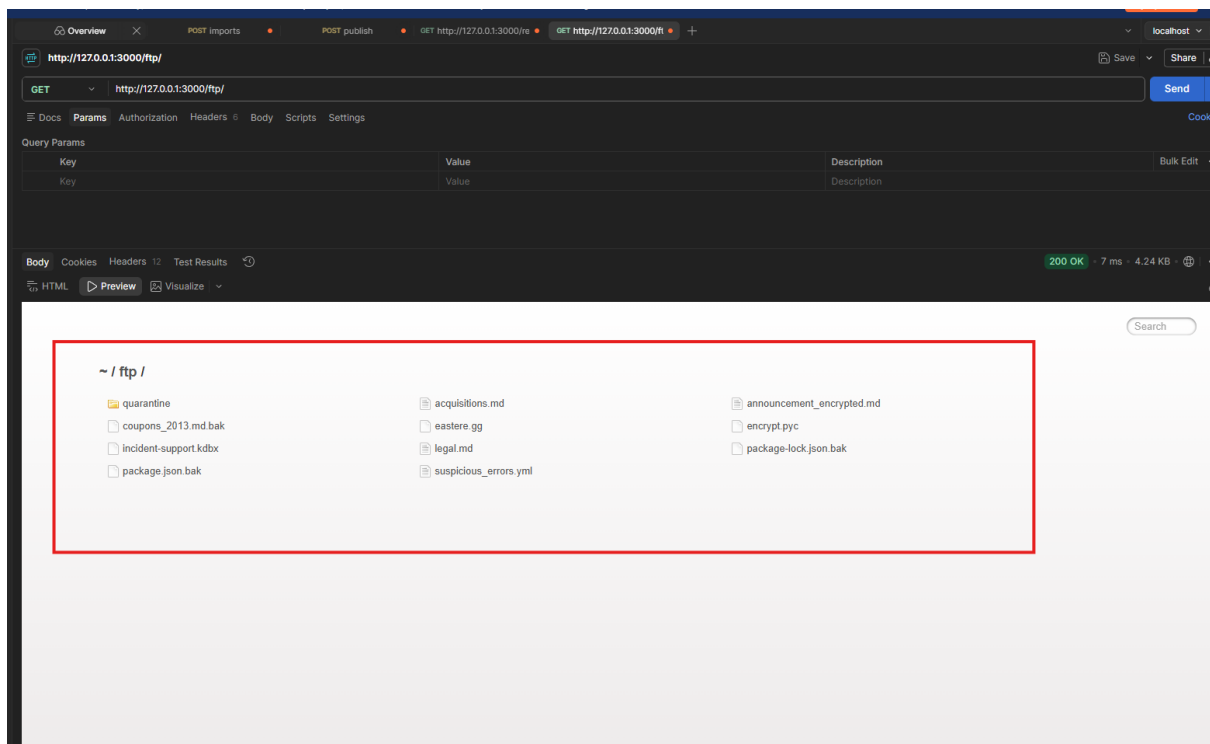


Figura 8 – Listagem de diretório de /ftp/ renderizada no Postman.

A Figura 9 mostra o conteúdo obtido ao requisitar diretamente `acquisitions.md`, com a frase de confidencialidade visível no corpo da resposta.

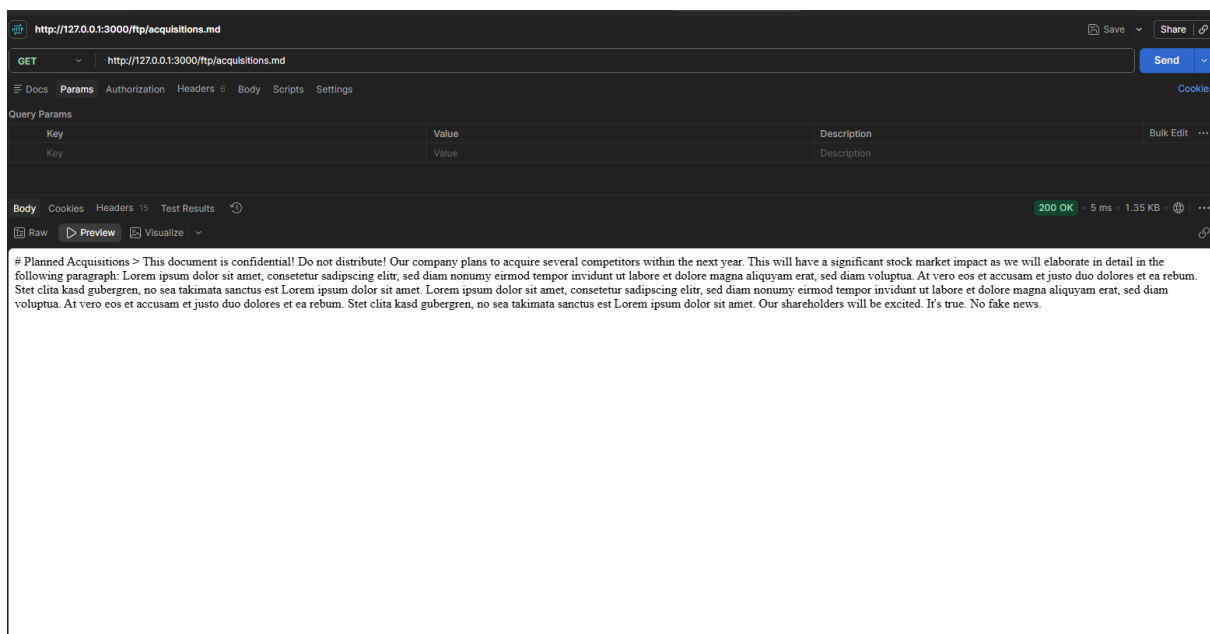


Figura 9 – Conteúdo de `acquisitions.md`, obtido sem autenticação, com a frase de confidencialidade visível no corpo da resposta.

A vulnerabilidade não está na existência de um arquivo com esse aviso, e sim na ausência de qualquer controle de acesso que impeça sua listagem e seu download

por um visitante não autenticado: o aviso de confidencialidade apenas evidencia o impacto da falha, já presente independentemente do conteúdo de cada arquivo.

7.6 V6, ausência de bloqueio contra força bruta no login (A07)

O endpoint de login aceita um número ilimitado de tentativas consecutivas com senha incorreta, sem qualquer atraso crescente, bloqueio temporário ou exigência de CAPTCHA.

Uma conta de teste foi submetida a quinze tentativas de login com senha incorreta, todas retornando HTTP 401 sem sinal de limitação. A tentativa seguinte, já com a senha correta, teve sucesso imediato, confirmando que nenhuma das quinze tentativas anteriores havia disparado algum mecanismo de defesa. A sequência completa está em evidencias/V6-bruteforce-login/http/, reproduzível com scripts/v6_bruteforce_login.py. Por se tratar de uma sequência de requisições repetitivas, não foi capturada como imagem: a evidência textual, com os dezesseis pares de requisição e resposta em ordem, demonstra o comportamento de forma completa.

7.7 V7, redefinição de senha via pergunta de segurança sem limite de tentativas (A04)

O fluxo de recuperação de senha do Juice Shop usa uma pergunta de segurança fixa por conta. Duas falhas de concepção se combinam: a pergunta associada a um e-mail pode ser consultada por qualquer pessoa, sem autenticação, e o endpoint que valida a resposta e efetiva a troca de senha não impõe limite de tentativas incorretas.

Uma conta de teste foi criada com uma resposta comum para a pergunta "Your eldest siblings middle name?". Consultando a pergunta associada ao e-mail sem qualquer autenticação e testando uma lista curta de cinco nomes prováveis, a quinta tentativa acertou a resposta, a senha foi redefinida e o login com a nova senha teve sucesso. Nenhuma das quatro tentativas incorretas anteriores provocou bloqueio. A sequência completa, incluindo a consulta da pergunta e as cinco tentativas de resposta, está em evidencias/V7-reset-senha-pergunta-seguranca/http/, reproduzível com scripts/v7_reset_senha_pergunta_seguranca.py.

Uma hipótese de escalada também foi testada e descartada: tentar associar uma resposta de segurança escolhida pelo atacante ao UserId de uma conta que já possuía resposta configurada, na expectativa de sequestrar contas alheias sem depender de adivinhação. A API rejeitou a tentativa (HTTP 400, restrição de unicidade em UserId), o que limita o achado ao cenário já demonstrado. O detalhamento desse teste negativo está na tabela de achados.

7.8 Categorias não confirmadas

A categoria A06 foi investigada, conforme descrito na Seção 7.4, sem produzir achado confirmado, por ausência de CVE aplicável à versão identificada. As categorias A08 (Software and Data Integrity Failures), A09 (Security Logging and Monitoring Failures) e A10 (Server-Side Request Forgery) permanecem sem investigação. A cobertura mínima de seis categorias já está atendida com A01, A02, A03, A04, A05 e A07; a investigação das três categorias restantes permanece como trabalho futuro, não como requisito pendente para a entrega mínima.

8. Conclusão

A priorização de correção segue a severidade de CVSS obtida para cada achado. Duas vulnerabilidades críticas dividem a prioridade máxima: a injeção SQL no login (V1) e a redefinição de senha via pergunta de segurança sem limite de tentativas (V7). Ambas comprometem isoladamente uma conta inteira, uma para qualquer conta da base e a outra para a conta administrativa, sem depender de nenhuma outra falha. Em seguida, a listagem de diretório (V5, alto) tem prioridade por expor um documento interno com uma correção de baixo custo, bastando desativar a listagem no servidor de arquivos estáticos. A exposição do hash de senha no token (V2), o controle de acesso quebrado na cesta (V3) e a ausência de bloqueio contra força bruta (V6) têm severidade média e exigem, respectivamente, remover dado de autenticação do payload do token, validar a posse do recurso no backend antes de retornar qualquer dado, e impor limite de tentativas com bloqueio temporário no login. A página de erro verbosa e a ausência de cabeçalhos de segurança (V4) têm a menor severidade entre os achados confirmados, mas amplificam o risco de outras falhas: a ausência de CSP, por exemplo, remove uma camada de contenção caso um XSS seja encontrado em investigação futura.

Um padrão comum a V1, V2 e V3 é que nenhuma delas foi descoberta por investigação livre: as três seguem exemplos ou pistas textuais do próprio roteiro do trabalho. Isso vale para V6 e V7, que partiram de exemplos citados nas seções de A07 e A04 do roteiro ("força bruta sem bloqueio" e "fluxo de recuperação de senha fraco"). O valor do exercício esteve em reproduzir cada achado com rigor, evidência em texto além da captura de tela, CWE validado diretamente na base do MITRE em vez de assumido pela categoria OWASP, e vetor CVSS justificado métrica a métrica. V4 e V5 exigiram investigação própria, a partir de sinais observados durante a preparação das ferramentas, e não de exemplos citados literalmente no roteiro. Em V7, a reprodução exigiu também engenharia reversa de um passo da API não documentado (a associação da resposta de segurança à conta), o que reforça que reproduzir um achado com rigor pode demandar investigação técnica própria mesmo quando a vulnerabilidade em si já é conhecida.

Com a cobertura mínima de seis categorias atendida, a conclusão desta etapa é que o Juice Shop, no estado avaliado, falha de forma consistente em três frentes: validação de entrada (V1), controle de acesso a nível de objeto (V3) e proteção dos mecanismos de autenticação e recuperação de conta (V2, V6, V7). Essas três frentes concentram os achados de maior severidade e devem orientar a priorização de correções caso o ambiente fosse levado a produção.

A conclusão definitiva, com a priorização final e as lições aprendidas do ciclo completo, será fechada após a investigação das categorias adicionais necessárias para atingir o mínimo de seis e a revisão conjunta da equipe.

9. Referências

OWASP Foundation. **OWASP Top 10:2021**. Disponível em: owasp.org/Top10.

OWASP Cheat Sheet Series; OWASP Application Security Verification Standard (ASVS). Disponível em: owasp.org.

FIRST. **CVSS, Common Vulnerability Scoring System, versão 3.1**. Disponível em: first.org/cvss.

MITRE. **CWE, Common Weakness Enumeration**. Disponível em: cwe.mitre.org.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais). Lei nº 12.737, de 30 de novembro de 2012 (tipificação de delitos informáticos).

Documentação oficial do OWASP Juice Shop. Disponível em: owasp.org.

Express.js. **Security updates**. Disponível em: expressjs.com/en/advanced/security-updates.

Apêndice A: Artefatos de evidência produzidos

Todos os artefatos citados neste relatório estão versionados no diretório evidencias/ do repositório. Os links abaixo apontam para a branch dev-diego; passam a resolver assim que essa branch for enviada ao repositório remoto.

Artefato	Descrição	Link
evidencias/README.md	Índice das evidências e convenção adotada para nomear pastas e arquivos	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/README.md
evidencias/image.png	Menu de conta no navegador, sessão autenticada como admin@juice-sh.op após o bypass de V1	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/image.png
evidencias/image-1.png	Requisição de login capturada no DevTools, com o payload de SQL Injection de V1	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/image-1.png
evidencias/image-2.png	Resposta do login capturada no DevTools, com o token retornado por V1	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/image-2.png
evidencias/image-3.png	JWT da conta administrativa decodificado em jwt.io, evidência de V2	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/image-3.png
evidencias/image-4.png	Resultado do fetch() no console do navegador, evidência de V3	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/image-4.png
evidencias/V1-sqli-login/http/	Par de requisição e resposta do bypass de login, gerado por scripts/v1_sqli_login.py	github.com/ThreeOneOneZero/vulnerable-web-application/tree/dev-diego/evidencias/V1-sqli-login/http/
evidencias/V1-sqli-login/ferramentas/01-sqlmap-login-email.txt	Saída completa do sqlmap confirmando a injeção booleana cega no parâmetro email e o SGBD SQLite	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/V1-sqli-login/ferramentas/01-sqlmap-login-email.txt
evidencias/V2-hash-senha-jwt/http/	Dois pares de requisição e resposta (registro e login da conta de teste), gerados por scripts/v2_hash_senha_jwt.py	github.com/ThreeOneOneZero/vulnerable-web-application/tree/dev-diego/evidencias/V2-hash-senha-jwt/http/

evidencias/V2-hash-senha-jwt/analise-jwt.txt	Payload do JWT decodificado e comparação entre o hash do token e o MD5 calculado localmente	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/V2-hash-senha-jwt/analise-jwt.txt
evidencias/V3-idor-cesta/http/	Seis pares de requisição e resposta (registro de duas contas, adição de item à cesta, leitura indevida), gerados por scripts/v3_idor_cesta.py	github.com/ThreeOneOneZero/vulnerable-web-application/tree/dev-diego/evidencias/V3-idor-cesta/http
evidencias/V4-erro-verboso-headers/http/ e .../ferramentas/	Requisição e resposta do erro verboso, e cabeçalhos da página inicial sem CSP/HSTS, capturados via curl	github.com/ThreeOneOneZero/vulnerable-web-application/tree/dev-diego/evidencias/V4-erro-verboso-headers
evidencias/V4-erro-verboso-headers/v4-erro-verboso.png	Captura de tela no Postman da página de erro renderizada, com a versão do Express exposta	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/V4-erro-verboso-headers/v4-erro-verboso.png
evidencias/V4-erro-verboso-headers/v4-headers-ausentes.png	Captura de tela no Postman dos cabeçalhos de resposta da página inicial, sem CSP nem HSTS	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/V4-erro-verboso-headers/v4-headers-ausentes.png
evidencias/V5-listagem-diretorio-ftp/http/	Listagem bruta de /ftp/ e os dois arquivos baixados sem autenticação, capturados via curl	github.com/ThreeOneOneZero/vulnerable-web-application/tree/dev-diego/evidencias/V5-listagem-diretorio-ftp/http
evidencias/V5-listagem-diretorio-ftp/v5-listagem-diretorio.png	Captura de tela no Postman da listagem de /ftp/ renderizada	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/V5-listagem-diretorio-ftp/v5-listagem-diretorio.png
evidencias/V5-listagem-diretorio-ftp/v5-documento-confidencial.png	Captura de tela no Postman do documento acquisitions.md, com o aviso de confidencialidade	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/V5-listagem-diretorio-ftp/v5-documento-confidencial.png
evidencias/V6-bruteforce-login/http/	Dezesseis pares de requisição e resposta (quinze tentativas incorretas e uma correta), gerados por scripts/v6_bruteforce_login.py	github.com/ThreeOneOneZero/vulnerable-web-application/tree/dev-diego/evidencias/V6-bruteforce-login/http
evidencias/V7-reset-senha-pergunta-seguranca/http/	Consulta da pergunta de segurança e as cinco tentativas de resposta até o acerto,	github.com/ThreeOneOneZero/vulnerable-web-application/tree/dev-

	gerados por scripts/v7_reset_senha_pergunta_seguranca.py	diego/evidencias/V7-reset-senha-pergunta-seguranca/http
evidencias/cvss_p1.png	Calculadora oficial do FIRST confirmando o vetor e o score de V1 e V7 (9.1, idêntico nos dois)	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/cvss_p1.png
evidencias/cvss_p2.png	Calculadora oficial do FIRST confirmando o vetor e o score de V2 isolado e V3 (6.5, idêntico nos dois)	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/cvss_p2.png
evidencias/cvss_v2_encadeado.png	Calculadora oficial do FIRST confirmando o vetor e o score de V2 encadeado a partir de V1 e de V5 (7.5, idêntico nos dois)	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/cvss_v2_encadeado.png
evidencias/cvss_v4.png	Calculadora oficial do FIRST confirmando o vetor e o score de V4 (5.3)	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/cvss_v4.png
evidencias/cvss_v6.png	Calculadora oficial do FIRST confirmando o vetor e o score de V6 (6.5)	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/evidencias/cvss_v6.png
relatorio/tabela-de-achados.md	Tabela consolidada dos achados e provas reprodutíveis completas de cada um, incluindo justificativa métrica do CVSS	github.com/ThreeOneOneZero/vulnerable-web-application/blob/dev-diego/relatorio/tabela-de-achados.md